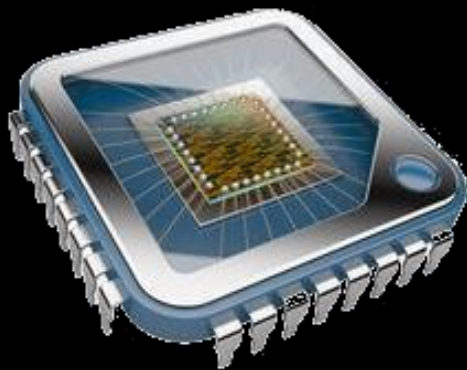




ORGANIZAÇÃO E ARQUITETURA DE COMPUTADORES

AULA 03: Código Assembly
Prof. Dr. Marcus

Conceitos aprendidos:

- O que é Assembly
- Relação direta com hardware
- Uso de registradores
- Instruções básicas
- Execução passo a passo

Linguagens de alto nível escondem o funcionamento interno do computador. Assembly mostra exatamente como o processador executa cada instrução.

1. O que significa x86-64?

O nome pode ser entendido em duas partes:

x86

Refere-se à família de processadores iniciada com o **Intel 8086** lançado em 1978.

Depois vieram versões compatíveis:

Intel 80286

Intel 80386

Intel 80486

Intel Pentium

Como todos terminavam com **86**, a arquitetura passou a ser chamada de **x86**.

1. O que significa x86-64?

64. Significa que o processador trabalha com **registradores de 64 bits**.

Isso permite:

- manipular números muito maiores
- acessar muito mais memória RAM
- executar programas mais complexos

A arquitetura foi criada inicialmente pela **AMD** no início dos anos 2000. Com nome original de **AMD64**.

Depois a **Intel** adotou o mesmo padrão. Hoje ambos usam essencialmente a mesma arquitetura.

2 - O que uma arquitetura de processador define?

Elemento	Descrição
Registradores	pequenas memórias dentro da CPU
Instruções	operações que o processador executa
Formato de dados	como números são armazenados
Endereçamento de memória	como acessar memória
Modos de operação	como o processador executa programas

Registradores na arquitetura x86-64

Registradores são memórias extremamente rápidas dentro da CPU.

Alguns dos principais registradores:

2 - O que uma arquitetura de FIAP

processador define?

Registradores na arquitetura x86-64

Registradores são **memórias extremamente rápidas dentro da CPU**.

Alguns dos principais registradores:

Registrador	Função
RAX	acumulador
RBX	registrador base
RCX	contador
RDX	dados auxiliares
RSP	ponteiro da pilha
RBP	base da pilha
RIP	ponteiro de instrução

Versões menores:

64 bits

32 bits

16 bits

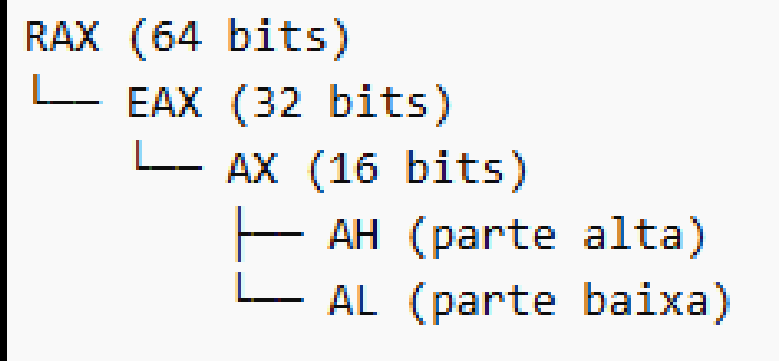
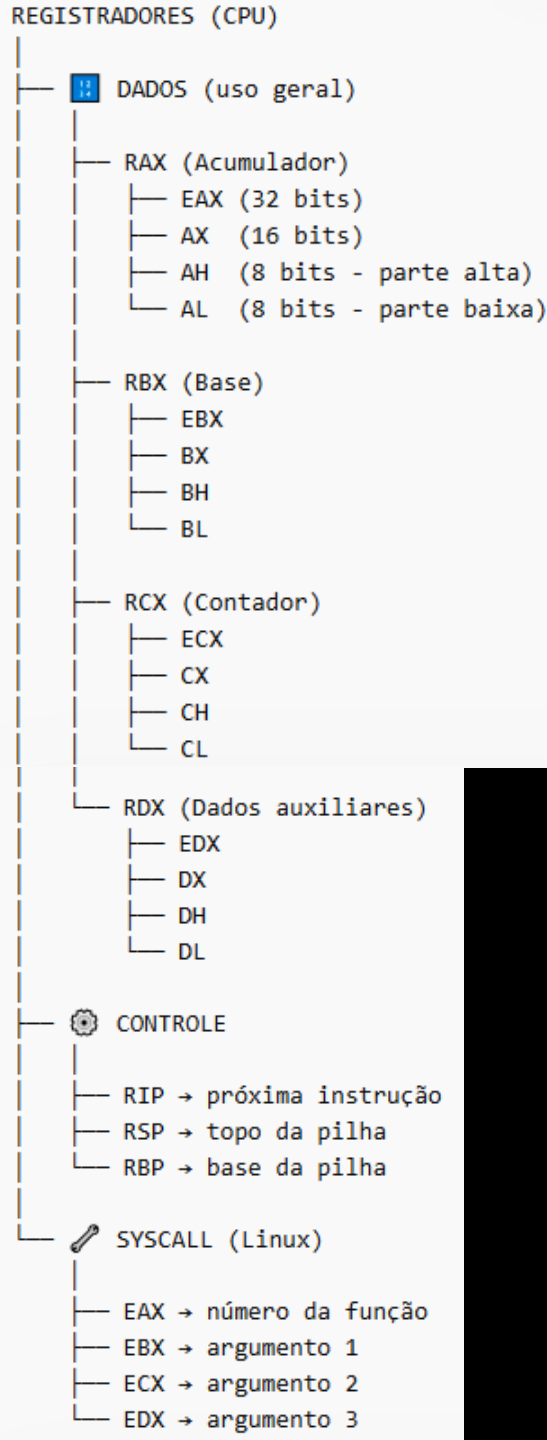
8 bits

RAX

EAX

AX

AL



3 - Instruções da arquitetura

A arquitetura x86-64 define **centenas de instruções**:

Exemplos básicos:

Instrução	Função
MOV	mover dados
ADD	somar
SUB	subtrair
INC	incrementar
DEC	decrementar
MUL	multiplicar
DIV	dividir

4 - Por que aprender x86-64 em

Arquitetura de Computadores?

Porque ela permite entender diretamente:

- funcionamento da CPU
- uso de registradores
- execução de instruções
- relação entre software e hardware

É exatamente o nível mais baixo antes da linguagem de máquina.

Fluxo de execução:

Programa em C



Compilador



Assembly



Código de máquina



CPU executa

5 – O que é um Assembler

Um Assembler é um programa que traduz linguagem Assembly → código de máquina.

Fluxo de execução de um programa:

Código Assembly



Assembler



Código de máquina (binário)



Processador executa

Exemplo em Assembly:

```
;</> assembly
```

```
mov eax, 5
```

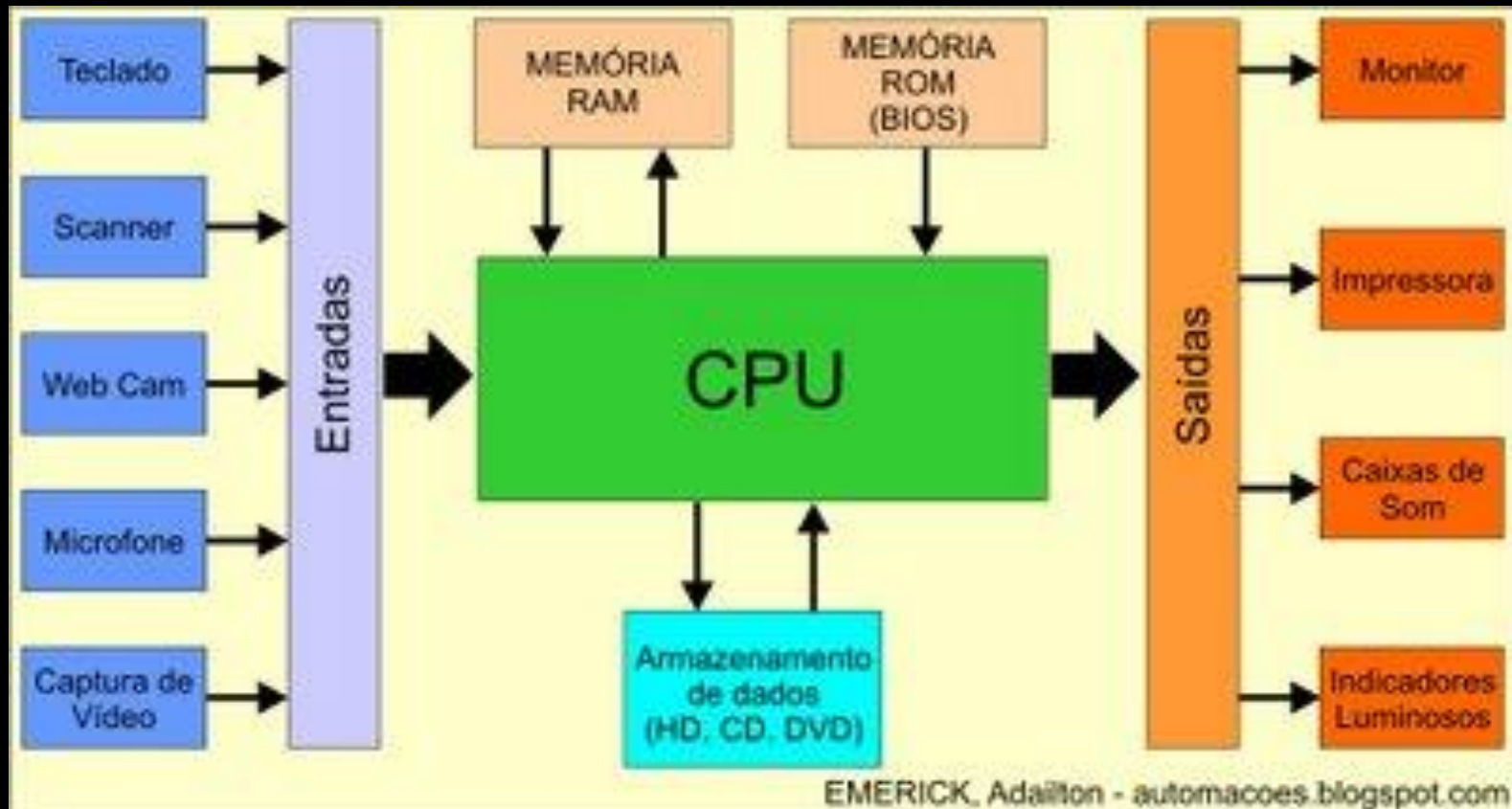
```
add eax, 3
```

Depois de montado pelo Assembler, isso vira **sequência de bytes binários** que o processador entende.

5 – CPU

Visão geral simples

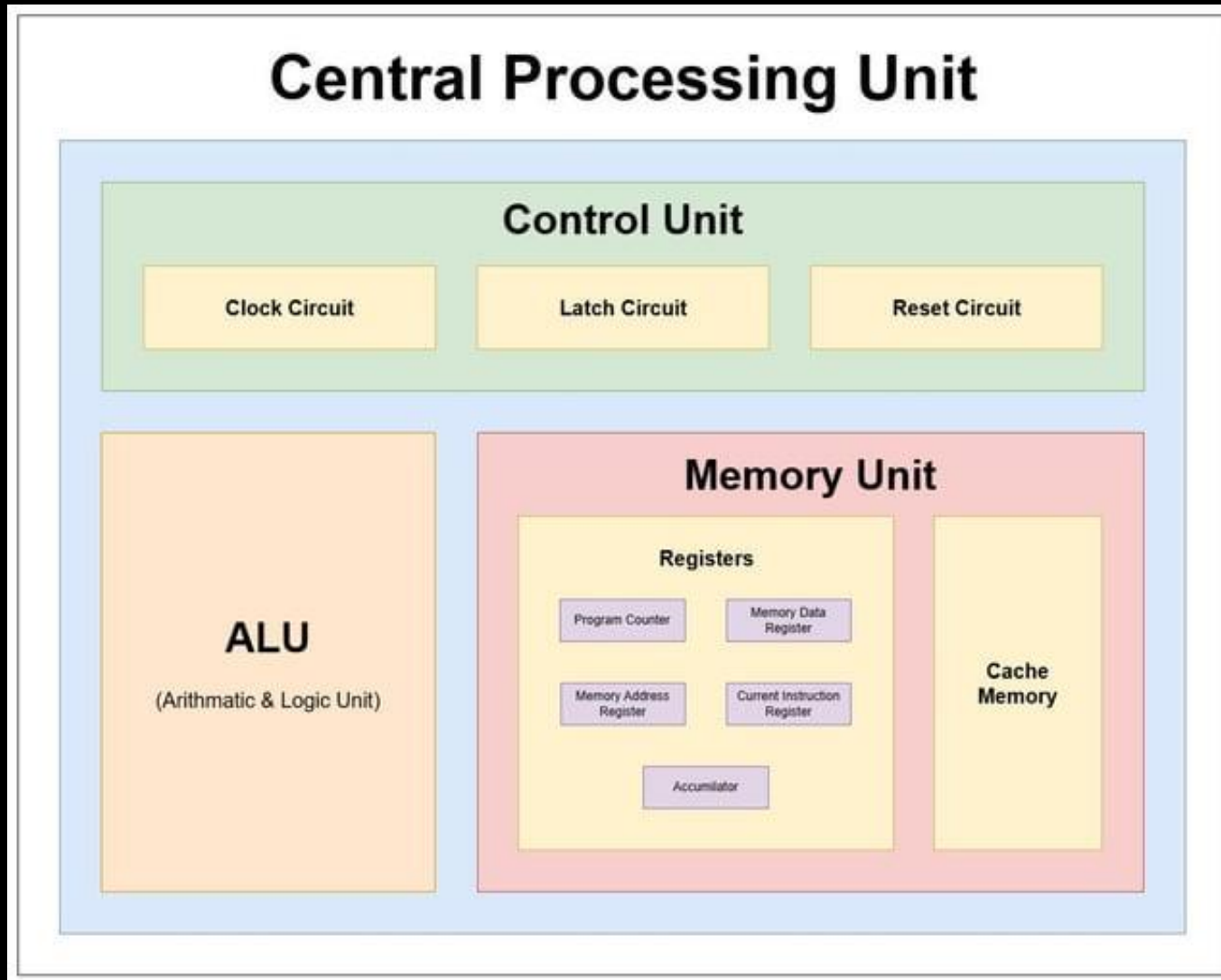
CPU = Unidade de Controle + ULA (ALU) + Registradores



5 – CPU

Visão geral simples

CPU = Unidade de Controle + ULA (ALU) + Registradores



5 – CPU

REGISTRADORES

O que são:

Pequenas memórias dentro da CPU (ultrarrápidas)

Função:

Guardar dados temporários

Guardar resultados

Armazenar operandos

**Guardar 5 dentro do registrador EAX
(dentro da CPU)**

Armazenados nos **registradores** ->

Tudo acontece: dentro da CPU

```
</> assembly
```

```
mov eax, 5
```

```
mov ebx, 3
```

Aqui:

```
EAX = 5
```

```
EBX = 3
```

5 – CPU

ULA (ALU — Unidade Lógica e Aritmética)

O que faz:

Realiza cálculos e operações lógicas

Exemplos:

- soma
- subtração
- multiplicação
- AND, OR

Exemplo

 `assembly`

```
add eax, ebx
```

 A ULA faz:

$$5 + 3 = 8$$

 Resultado volta para:

EAX = 8

UNIDADE DE CONTROLE

O que faz:

Controla toda execução do programa

Responsável por:

Ler instruções

Decodificar

Mandar executar

Controlar fluxo

Exemplo

```
</> assembly
```

```
mov eax, 5
```

```
add eax, 3
```

 A Unidade de Controle:

1. busca a instrução
2. entende ("mov", "add")
3. manda a ULA executar
4. decide a próxima instrução

5 – CPU

RESUMINDO

O que acontece:

1. Controle lê instrução
2. Registradores recebem valores
3. ULA faz a soma
4. Resultado volta para EAX

Fluxo completo

Memória → CPU → Registradores → ULA → Registradores → Saída

```
</> assembly
```

```
mov eax, 5  
mov ebx, 3  
add eax, ebx
```

[ROM] → inicialização

[RAM] ↔ [CACHE] ↔ [CPU]

- Registradores
- ULA
- Controle



OBJETIVO DA AULA

Os alunos serão capazes de:

- multiplicar e dividir números em Assembly
- entender onde ficam os resultados (AL, AH, AX)
- mostrar o resultado na tela (1 ou 2 dígitos)

`</>` assembly

AL → número principal

BL → segundo número

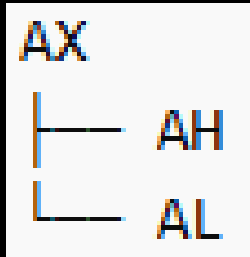
E:

div → divide

mul → multiplica

Exemplo 1

```
mov al, 17
mov bl, 10
div bl
```



Instrução	AL	AH	BL
mov al,17	17	?	?
mov bl,10	17	?	10
div bl	1	7	10

Resultado:

AL = 1 (dezena)
AH = 7 (unidade)

Saída

AX : BL → resultado em AX

REGISTRADORES

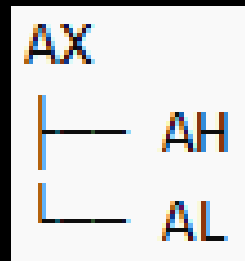
AL → Quociente

AH → resto da divisão

BL → divisor

CPU sempre usa AX por regra da instrução DIV. Já foi projetada pela arquitetura para sempre armazenar o resultado em AX

7 – Div



Exemplo 2

```
mov al, 9
mov bl, 2
div bl
```

Internamente a
CPU vê assim:
 $AX \div BL$

REGISTRADORES

AL → Quociente

AH → resto da divisão

BL → divisor

Mas...

you só colocou valor em AL
então:

```
AX = [ AH | AL ]
      [  0 |  9 ]
```

```
BL = 2
```

O programador NÃO escolhe usar AX

A instrução div já foi projetada assim:
Ela sempre usa AX automaticamente

Depois da divisão

```
AX = [ AH | AL ]
      [ 1 | 4 ]
```

Pergunta

```
mov al, 20  
mov bl, 3  
div bl
```

O que está em AX antes?

Quanto dá $20 \div 3$?

RESUMO GERAL

DIV sempre faz:

$AX \div \text{registrador}$

Resultado:

AL $\rightarrow$ quociente

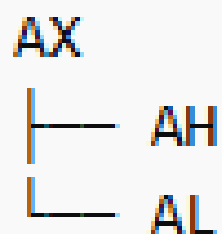
AH $\rightarrow$ resto

8 – Mul

Exemplo 1

Multiplicação usa AX (não só AL)

```
mov al, 4  
mov bl, 3  
mul bl
```



Resultado:

AL = 4 BL = 3
AH = ?

ANTES:

AL = 4

BL = 3



MUL BL



DEPOIS:

AX = 12

[AH | AL]

[0 | 12]

A CPU entende automaticamente:

AL × BL →
resultado em
AX

Por que usa AX?

Porque o resultado pode ser maior que 255

Então precisa de:

16 bits (AX)

Importante

```
mul bl
```

👉 NÃO significa:

```
bl × bl ❌
```

👉 significa:

```
AL × BL
```

8 – Mul

Exemplo 2

```
mov al, 20  
mov bl, 20  
mul bl
```

Resultado:

$20 \times 20 = 400$

AX = 400

Separando:

AH = 1

AL = 144

Exemplo 3

```
mov al, 5  
mov bl, 4  
mul bl
```

Pergunte:

Qual o Resultado?

Onde fica?

RESUMO GERAL

MUL pega o valor do registrador principal,
multiplica pelo operando informado
e coloca o resultado em um
registrador maior.

```

1 section .data
2     msg db "Multiplicacao: "
3     tam equ $-msg
4
5 section .bss
6     dez resb 1
7     uni resb 1
8
9 section .text
10    global _start
11
12 _start:
13
14    ; ----- MULTIPLICAÇÃO -----
15    mov al, 4
16    mov bl, 3
17    mul bl           ; AX = 12
18
19    ; ----- SEPARAR DÍGITOS -----
20    mov bl, 10
21    div bl           ; AL = dezena, AH = unidade
22
23    mov [dez], al
24    mov [uni], ah
25

```

```

; ----- ASCII -----
add byte [dez], 48
add byte [uni], 48

; ----- PRINT -----
mov eax,4
mov ebx,1
mov ecx,msg
mov edx,tam
int 0x80

; dezena
mov eax,4
mov ebx,1
mov ecx,dez
mov edx,1
int 0x80

; unidade
mov eax,4
mov ebx,1
mov ecx,uni
mov edx,1
int 0x80

; exit
mov eax,1
int 0x80

```



PARTE 3 — DIVISÃO



Como funciona o `div`

```
</> assembly
```

```
div bl
```



CPU faz:

$AL \div BL$



Exemplo

```
</> assembly
```

```
mov al, 9
```

```
mov bl, 2
```

```
div bl
```

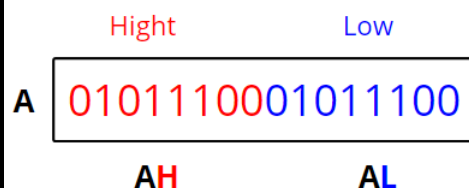
Resultado:

$AL = 4$

$AH = 1$

`mul` → multiplica e guarda em AX

`div` → divide e separa em AL (resultado) e AH (resto)

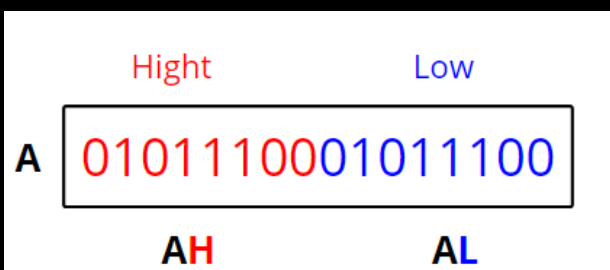


Comparação

Operação	Entrada	Saída
soma	AL, BL	AL
multiplicação	AL, BL	AX
divisão	AL, BL	AL + AH

`mul` → multiplica e guarda em AX

`div` → divide e separa em AL (resultado) e AH (resto)



ATIVIDADES

1 – Dado o número inicial 4. Aplicar INC, INC (Incremento duplo), desenvolva um programa em Assembly para mostrar o resultado esperado.

2 – Desenvolver os seguintes cálculos em código Assembly

(a) $5 \times 6 = ?$

(b) $8 \div 3 = ?$

(c) $7 \times 8 = ?$

(d) $12 + 7 = ?$